

Log In ▾

This page is under active construction. Expect bugs.

Techniky pro efektivní implementaci uživatelského rozhraní. Příprava uživatelského rozhraní pro testování s/bez uživatele.

vector2 ▾

B4B39IUR [Webové stránky předmětu](#)

- **Techniky pro efektivní implementaci uživatelského rozhraní:**

1. **Implementace MVVM modelu** – pomocí návrhových vzorů (např. observer, event-delegate, publish-subscribe). Řešení vztahu View-ViewModel pomocí návrhového vzoru "Data Binding".
2. **Přizpůsobení (customization) UI komponent** – pomocí šablon (DataTemplate, ControlTemplate), stylů a triggerů. Vytváření tzv. User Control a Custom Control.
3. **Validace uživatelského vstupu** – pomocí validačních pravidel a interfaců (Data source exception, data error interface), případně vlastní validační třídou. Prezentace chyb pomocí šablon a triggerů.

- **Příprava uživatelského rozhraní pro testování s/bez uživatele:**

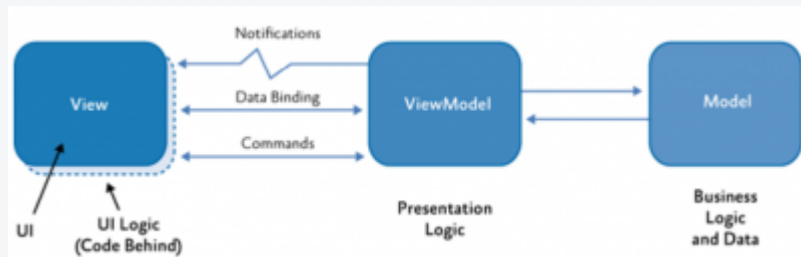
1. **Postupy pro důsledné oddělení jednotlivých částí software** – podle vzoru MVC/MVP/MVVM. Vytváření UI test skriptu (testovací kód, nahrávání interakce, GUI ripping).
2. **Příprava software pro uživatelské testování** – data mocking, podpora metody Wizard-of-Oz, sběr dat z chování software a interakce s uživatelem.

1. Techniky pro efektivní implementaci uživatelského rozhraní

Implementace MVVM modelu

Model–View–ViewModel (MVVM) je architektonický vzor navržený pro moderní GUI aplikace (zejména WPF, .NET MAUI, Xamarin). Jeho hlavní výhodou je důsledné oddělení prezentační logiky od vzhledu (UI), což usnadňuje údržbu, testování a opětovné použití kódu.

- **Model** – Obsahuje samotná data a logiku aplikace. Může zahrnovat přístup k databázi, validaci či výpočty. Je zcela nezávislý na UI.
- **ViewModel** – Představuje most mezi UI a Modelem. Obsahuje data upravená pro View, implementuje `INotifyPropertyChanged` pro upozornění na změny a definuje příkazy (`ICommand`) pro reakce na uživatelské akce.
- **View** – Zobrazuje data z ViewModelu pomocí bindingu, neobsahuje žádnou aplikační logiku. Může být definována v XAML, přičemž `DataContext` odkazuje na odpovídající ViewModel.



Klíčové návrhové vzory použité v MVVM

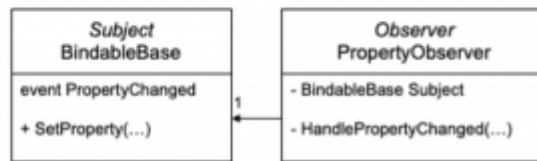
- **Observer Pattern**

- Definuje *jeden ku mnoha* závislost: když se subjekt (např. Model) změní, informuje všechny zaregistrované pozorovatele (např. ViewModel). Tento vzor pomáhá oddělit Model od View/ViewModelu, čímž se zvyšuje modularita. V C# se často realizuje pomocí `INotifyPropertyChanged`.

```

public class HelloBoundWorldViewModel : BindableBase {
    private string firstName;
    public string FirstName
    {
        get { return firstName; }
        set { SetProperty(ref firstName, value); }
    }
}

```

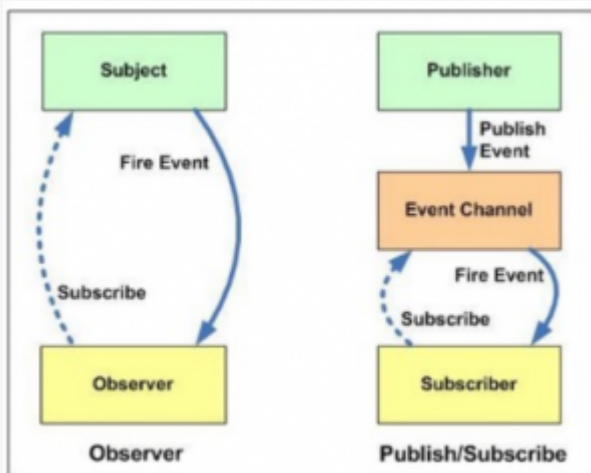


• Event–Delegate Mechanismus (C#)

- Delegáty a eventy implementují observer pattern. `event` je seznam metod (delegátů), které jsou spuštěny při určité události (např. změna hodnoty).
- `delegate` definuje typ metody pro zpracování události
- `event` reprezentuje událost (např. kliknutí na tlačítko)
- `EventHandler` a `PropertyChangedEventArgs` jsou běžně používané standardizované typy

• Publish–Subscribe Pattern

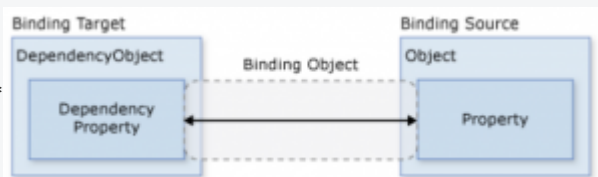
- Zajišťuje volné provázání komponent: *publishers* posílají zprávy, *subscribers* se přihlásí k typům zpráv, které je zajímají. Obě strany o sobě navzájem neví. To usnadňuje rozšiřitelnost a škálovatelnost, i když složitost správy zpráv může být vyšší.

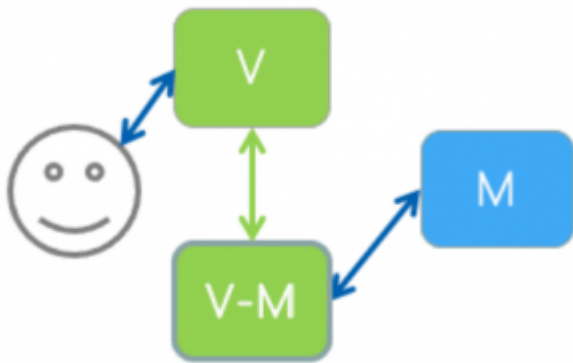


• Data Binding Pattern

Umožňuje synchronizaci dat mezi View a ViewModel. Implementuje se pomocí:

- XAML bindingu (`{Binding propertyName}`)
- `INotifyPropertyChanged` (vlastnosti ve ViewModelu vyvolávají notifikace)
- `DataContext` – nastavuje výchozí zdroj dat pro binding v UI





• Command Pattern (`ICommand`)

- Abstrahuje uživatelské akce do objektů – například kliknutí na tlačítko nespouští metodu přímo, ale přes `ICommand`.
- `Execute()` definuje, co se má stát
- `CanExecute()` určuje, zda je příkaz aktuálně povolen
- Často se používá `RelayCommand` nebo `DelegateCommand` pro jednoduchou implementaci bez nutnosti psát nové třídy pro každou akci

Přizpůsobení UI komponent

Přizpůsobení (customization) UI komponent umožňuje upravit vzhled i chování ovládacích prvků tak, aby odpovídaly požadavkům aplikace, designéra nebo platformy. Díky tomu lze dosáhnout konzistentního vizuálního stylu, přístupnosti i přívětivosti pro uživatele. V prostředí WPF nebo MAUI se přizpůsobení realizuje pomocí stylů, šablon, triggerů a dvou typů komponent: `UserControl` a `CustomControl`.

Styly a triggery

• Style

- Style je kolekce `Setter` prvků, která definuje vzhled komponenty – např. barvu pozadí, font, velikost nebo jiné vizuální vlastnosti.
- Umožňuje oddělit vzhled od logiky, podobně jako CSS v HTML.
- Každý `Style` může být uložen jako `StaticResource` a znovu použit na více prvcích.

• Triggers

- Triggery umožňují změnit vlastnosti komponenty na základě určité podmínky.
- Typy triggerů:
 1. `PropertyTrigger`: aktivuje se při změně vlastnosti (např. `IsMouseOver == true`).
 2. `DataTrigger`: aktivuje se při změně vázaných dat (např. hodnota z ViewModelu).
 3. `EventTrigger`: reaguje na události (např. kliknutí), často spouští animace.
- Když přestanou platit podmínky triggeru, změny se automaticky vrací zpět.

Šablony (Templates)

• DataTemplate

- Definuje vizualizaci datového objektu – např. jak bude vypadat každá položka v `ListBoxu`.
- Umožňuje změnit strom vizuálních prvků zobrazených pro konkrétní datovou instanci.
- Využívá se typicky pro seznamy (`ItemsControl`, `ListView`, `ComboBox`).

• ControlTemplate

- Umožňuje kompletně změnit vzhled a strukturu ovládacího prvku, ale zachovává jeho funkčnost.
- Nahrazuje celý strom komponenty (např. tlačítko může být vykresleno jako kruh).
- Používá `TemplateBinding` pro přístup k hodnotám vlastností rodičovského prvku.

• ContentTemplate

- Používá se u prvků jako `ContentControl` (např. `Button`), když chceme vnořit složitější obsah (např. obrázek + text).
- Kombinuje různé prvky (např. `Image`, `TextBlock`) a určuje, jak se data mají zobrazit.

Přizpůsobení komponent: UserControl vs CustomControl

• UserControl

- Kombinuje více existujících ovládacích prvků do jednoho bloku.
- Používá se, pokud potřebujeme opakovat určitý kus UI napříč aplikací.
- Implementace je snadná (XAML + code-behind), ale omezená: nelze stylovat pomocí `ControlTemplate`.

• CustomControl

- Dědí z existující komponenty (např. `Button`) nebo vytváří zcela novou.
- Umožňuje stylování, šablonování i override chování – plná kontrola nad vzhledem a interakcí.
- Používá se pro tvorbu opakovatelně použitelných, plně stylovatelných komponent.
- Vyžaduje definici ve `Themes/Generic.xaml` + třídu v C#.

Ukázky použití (XAML)

Tlačítko s obrázkem a textem:

```
<Button HorizontalAlignment="Center" VerticalAlignment="Center">
  <StackPanel Orientation="Horizontal">
    <Image Source="icon.png" Height="32" Width="32"/>
    <TextBlock Text="Settings" VerticalAlignment="Center"/>
  </StackPanel>
</Button>
```

Styl se StaticResource:

```
<Window.Resources>
  <Style x:Key="ButtonStyle" TargetType="Button">
    <Setter Property="Background" Value="LightBlue"/>
    <Setter Property="FontWeight" Value="Bold"/>
  </Style>
</Window.Resources>
<Button Style="{StaticResource ButtonStyle}" Content="Styled Button"/>
```

Trigger na `IsMouseOver`:

```
<Style TargetType="Button">
  <Style.Triggers>
    <Trigger Property="IsMouseOver" Value="True">
      <Setter Property="Background" Value="Yellow"/>
    </Trigger>
  </Style.Triggers>
</Style>
```

Custom seznam s `DataTemplate`:

```
<ListBox>
  <ListBox.ItemTemplate>
    <DataTemplate>
      <StackPanel>
        <TextBlock Text="{Binding Name}"/>
        <TextBlock Text="{Binding Description}"/>
      </StackPanel>
    </DataTemplate>
  </ListBox.ItemTemplate>
</ListBox>
```

Použití `ContentTemplate` v tlačítku:

```
<Button>
  <Button.ContentTemplate>
    <DataTemplate>
      <StackPanel Orientation="Horizontal">
        <Image Source="{Binding Image}" Width="16" Height="16"/>
        <TextBlock Text="{Binding Text}"/>
      </StackPanel>
    </DataTemplate>
  </Button.ContentTemplate>
</Button>
```

Skins a Themes

• Skins

- Kolekce stylů pro celou aplikaci. Umožňuje dynamicky měnit vzhled za běhu aplikace.
- Používá `DynamicResource` a přepínání zdrojů ve `ResourceDictionary`.

- **Themes**

- Respektují systémové nastavení (např. dark/light mode).
- Používají se pro přístupnost, škálování, barevné schéma apod.

Validace uživatelského vstupu

Validace vstupu zajišťuje, že data zadaná uživatelem odpovídají očekávanému formátu, rozsahu či logickým podmínkám. WPF nabízí robustní validační mechanismy přímo integrované do systému data bindingu.

Metody validace

- **ExceptionValidationRule**

- Validace probíhá tak, že se při aktualizaci datové vlastnosti očekává možná výjimka – pokud k ní dojde, binding ji zachytí a označí vstup jako chybný.
- Vhodné pro základní typové validace, např. převod ``string`` na ``int``.

- **IDataErrorInfo / INotifyDataErrorInfo**

- Implementace těchto rozhraní ve ViewModelu umožňuje kontrolu jednotlivých vlastností a vracení chybových zpráv.
- ``IDataErrorInfo`` umožňuje jednoduchou synchronní validaci pomocí indexeru ``this[string propertyName]``.
- ``INotifyDataErrorInfo`` podporuje i asynchronní validaci a vícenásobné chyby na jedné vlastnosti – hodí se pro složitější scénáře.

- **Custom ValidationRule**

- Vytvořením vlastní třídy, která dědí z ``ValidationRule``, lze implementovat složitější validační logiku.
- Třída implementuje metodu ``Validate``, která vrací ``ValidationResult``, obsahující boolean a případné chybové hlášení.
- Umožňuje znovupoužití pravidla pro více polí nebo v různých částech aplikace.

Zobrazení chyb

- **ErrorTemplate**

- Speciální ``ControlTemplate``, který se použije při zobrazení chyby. Typicky obsahuje vizuální zpětnou vazbu (červený rámeček, ikona, tooltip).
- Uvnitř šablony se používá ``AdornedElementPlaceholder``, který nahrazuje původní prvek a zobrazuje chybu ve stejné pozici.

- **Style.Triggers na `Validation.HasError`**

- Pomocí ``Trigger`` lze změnit vzhled komponenty při výskytu chyby – např. zobrazit tooltip s chybou, změnit barvu okraje nebo pozadí.
- Dynamická prezentace chybových hlášek: např. bindingem na ``Validation.Errors[0].ErrorContent``.

Principy použitelné validace

- Validace má být **srozumitelná** – uživatel musí pochopit, co je špatně a jak to opravit.
- Má být **neinvazivní** – chyba by neměla přerušit práci, ale měla by být patrná a opravitelná.
- Validace by měla být **integrovaná do UI** – přímo u daného vstupu, nikoliv jako oddělené hlášení.

Ukázky použití

Validace pomocí výjimek

```
<TextBox.Text>
  <Binding Path="Age">
    <Binding.ValidationRules>
      <ExceptionValidationRule />
    </Binding.ValidationRules>
  </Binding>
```

Validace pomocí ``IDataErrorInfo``

```
<TextBox Text="{Binding Name, ValidatesOnDataErrors=True}" />
```

Vlastní validační pravidlo

```

public class JpgValidationRule : ValidationRule {
    public override ValidationResult Validate(object value, CultureInfo cultureInfo) {
        string path = value as string;
        if (!path.EndsWith(".jpg"))
            return new ValidationResult(false, "Soubor musí být ve formátu JPG.");
        return new ValidationResult(true, null);
    }
}

```

Zobrazení chyby pomocí Triggeru

```

<Style TargetType="TextBox">
    <Style.Triggers>
        <Trigger Property="Validation.HasError" Value="true">
            <Setter Property="ToolTip"
                Value="{Binding RelativeSource={RelativeSource Self},
                    Path=(Validation.Errors)[0].ErrorContent}" />
        </Trigger>
    </Style.Triggers>
</Style>

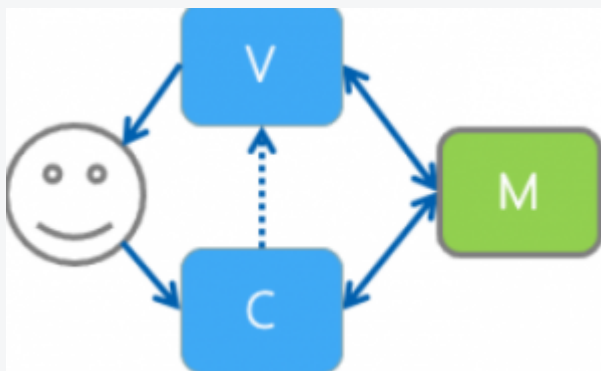
```

2. Příprava uživatelského rozhraní pro testování s/bez uživatele

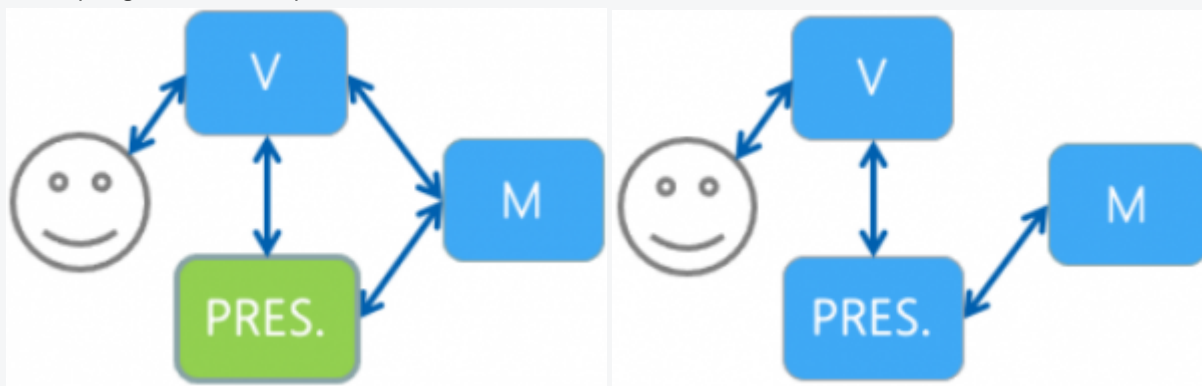
Postupy pro důsledné oddělení částí systému

- **Architektury:**

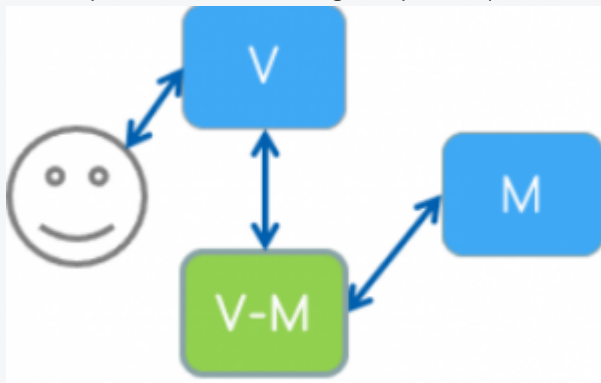
- **MVC** – klasické oddělení logiky, prezentace a dat, uživatel interaguje i s View, Controller zpracovává vstup a mění stav Modelu; View ho pouze zobrazuje. Často dochází k prolínání zodpovědností mezi View a Controllerem.



- **MVP** – presenter obsluhuje interakci s uživatelem a logiku, View je pasivní, vhodné pro testovatelnost, ale tight-coupling může ztížit správu.



- **MVVM** – silný binding, ViewModel poskytuje logiku, ViewModel je zcela oddělený od View. Spojení je dosaženo pomocí *data bindingu* a příkazů, což usnadňuje testování i vývoj.



- **UI testovací skripty:**

- **Testovací kód (UI Test Scripts)** – Skripty automatizují interakce uživatele s UI (např. kliknutí, vyplnění polí) a ověřují očekávané chování aplikace. Test je obvykle strukturován s identifikátorem, účelem, vstupy, očekávaným výstupem a historií provedení.
 - Ukázka v Selenium:

```
public void testLogin() throws Exception {  
    selenium.open("/MyApp/");  
    selenium.type("name=username", "tester");  
    selenium.type("name=password", "1234");  
    selenium.click("name=login");  
    selenium.waitForPageToLoad("3000");  
}
```

- **Nahrávání interakce** – Nástroje jako Barista nebo Selenium IDE umožňují zaznamenat skutečné akce uživatele a automaticky z nich vytvořit testovací skripty. Zjednodušuje tvorbu testů a snižuje množství chyb způsobených ručním psaním.
- **Automatizované testování (Automated replay)** – Testovací nástroje (např. Selenium, Firebase Test Lab) spouštějí zaznamenané nebo předem definované testy automaticky. Výstupem může být:
 - počet zobrazených obrazovek,
 - snímky obrazovky (screenshots),
 - provedené akce (click, input, scroll...),
 - záznam výjimek a chyb,
 - metriky jako doba odezvy nebo dostupnost komponent.
- **Manuální testování** – Tester ručně následuje předem definované kroky a porovnává výstup s očekáváním. Hodí se pro testování na různých zařízeních nebo pro explorativní testy.
- **Model-based testing** – Na základě vytvořeného modelu UI (např. stavový automat nebo hierarchie obrazovek) se automaticky generují testovací scénáře. Pomáhá pokrýt všechny důležité přechody a stavy v aplikaci.
- **GUI ripping** – Automatická analýza uživatelského rozhraní, která vytvoří hierarchický model UI (strom komponent), určí dostupné akce a jejich předpoklady. Nástroj poté generuje testovací scénáře pro různé kombinace uživatelských operací.
 - Např. operátor `File\_Open("public", "doc.doc")` → otevření souboru ve složce „public“.
 - Plánovací algoritmus vytvoří posloupnost interakcí, která ověří, že cesta od výchozího stavu k cíli je funkční.
- **Porovnání návrhu a implementace** – Pomocí nástrojů jako Diff Checker lze porovnat očekávaný návrh UI (např. prototyp z Figma) s implementovaným rozhraním a vyhodnotit rozdíly. Toto se využívá hlavně při regresech a kontrolách konzistence vzhledu.

Příprava software pro uživatelské testování

Aby bylo možné efektivně provádět testování s uživateli (nebo bez nich), je potřeba připravit samotné prostředí – od simulace dat přes zachytávání interakcí až po nástroje pro pokročilou analýzu chování.

Data mocking

- Data mocking nahrazuje reálná data fiktivními, čímž umožňuje testování v situacích, kdy ještě neexistuje backend, API nebo validní datové sady.
- Lze využít k simulaci specifických scénářů – např. chybových stavů, prázdných vstupů, nestandardního chování systému.
- Umožňuje testovat UI nezávisle na aktuálním stavu backendu (lo-fi i hi-fi prototypy).

Wizard-of-Oz metoda

- Tato metoda spočívá v tom, že části systému, které ještě nejsou implementovány (např. hlasové rozpoznání), jsou skrytě obsluhovány člověkem.
- Uživatel je často přesvědčen, že interaguje s plně funkčním systémem – získáme tak autentickou zpětnou vazbu na nové koncepty a interakce.
- Použití např. u testování dialogových agentů, asistentů do aut nebo komplexních rozhraní.
- Umožňuje testovat i náročné technologie bez nutnosti jejich okamžité realizace.

Sběr dat z testování

- **Základní metody:**
 - Videozáznam (1st-person, 3rd-person, screen recording)
 - Poznámky pozorovatele
 - Dotazníky před/po testování
- **Pokročilé metody:**
 - **Logy** – záznam interakcí v systému (např. kliknutí, vstupy)
 - **Eye-tracking** – sledování fixace očí, pozornosti uživatele
 - **Senzory** – biometrie, prostředí
 - **KLM model (Keystroke-Level Model)** – predikce doby interakce na základě základních operací (klik, psaní, mentální pauzy)
 - **Android Monkey** – náhodné generování uživatelských akcí pro zátěžové testování

Testování bez uživatele

- Prováděno expertem bez přítomnosti reálného uživatele. Výhodou je rychlost a možnost aplikace v raných fázích vývoje.
- **Heuristická evaluace:**
 - Hodnocení použitelnosti podle souboru heuristik (např. Nielsen – viditelnost stavu systému, konzistence, prevence chyb...).
 - Vyžaduje více hodnotitelů – každý nezávisle identifikuje problémy a při konsolidaci se stanovuje závažnost.
- **Kognitivní průchod (Cognitive Walkthrough):**
 - Metoda simulující mentální proces uživatele – „co si myslí, že by měl udělat?“.
 - Krok po kroku ověřuje, zda je rozhraní srozumitelné a odpovídá cílům uživatele.
 - Klíčové otázky:
 - Q0: Co chce uživatel dosáhnout?
 - Q1: Bude správná akce zřejmá?
 - Q2: Bude akce odpovídat uživatelskému záměru?
 - Q3: Dostane uživatel srozumitelnou zpětnou vazbu?

Co testovat

- **Funkčnost** – fungují prvky správně, jak bylo navrženo?
- **Dostupnost komponent** – jsou všechny prvky dosažitelné (např. z pohledu přístupnosti)?
- **Dokončení úkolů** – lze úlohy dokončit s přiměřeným úsilím?
- **Chybovost** – kde může dojít ke zmatku nebo chybě?
- **Čitelnost a naležitelnost** – lze rychle najít potřebné informace nebo ovládací prvky?
- **Použitelnost** – celková přívětivost, subjektivní spokojenost, intuitivnost rozhraní.

Trace: • [b4b39iur](#)